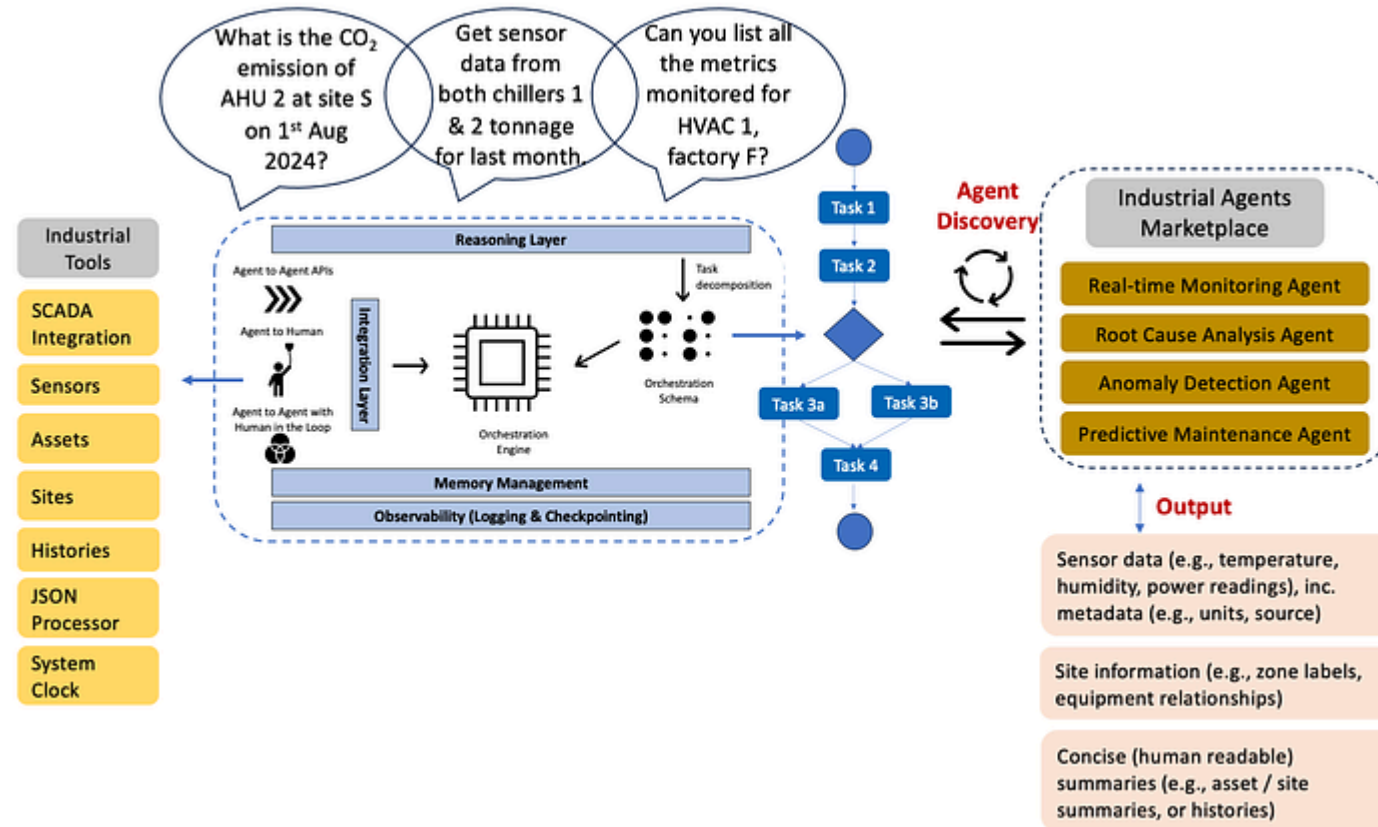


[< Go to the original](#)

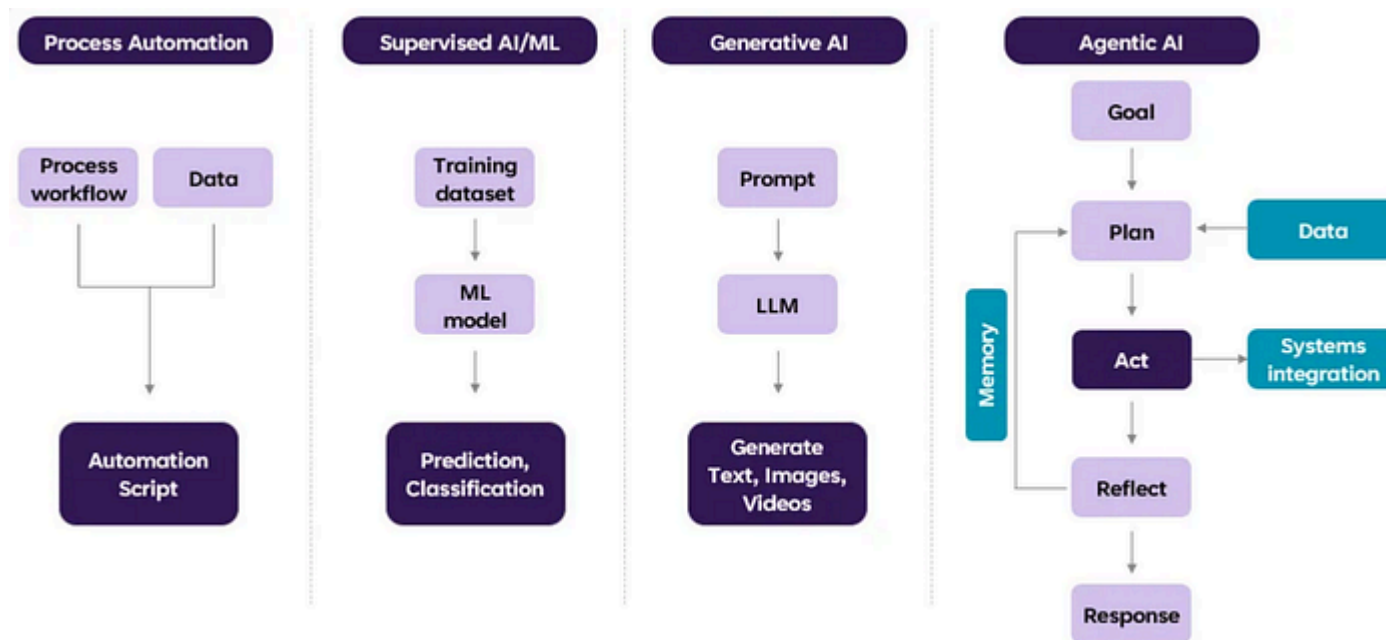
Agentic AI Reasoning for Industrial IoT

Reinventing Manufacturing processes using autonomous AI Agents

Debmalya Biswas

1. Introduction

The discussion around ChatGPT (in general, generative AI), has now evolved into agentic AI. While ChatGPT is primarily a chatbot that can generate text responses, AI agents can execute complex tasks autonomously, e.g., make a sale, plan a trip, make a flight booking, book a contractor to do a house job, order a pizza. Fig. 1 below illustrates the evolution of agentic AI systems.



Bill Gates recently envisioned a future where we would have an AI agent that is able to process and respond to natural language and accomplish a number of different tasks. Gates used planning a trip as an example.

Ordinarily, this would involve booking your hotel, flights, restaurants, etc. on your own. But an AI agent would be able to use its knowledge of your preferences to book and purchase those things on your behalf.

The key characteristics of agentic AI systems are their **autonomy** and **reasoning** prowess that allow them to decompose complex tasks into smaller executable tasks, and then orchestrate their execution in a way that can monitor, reflect, and adapt / self-correct the execution as and when needed. Given this,

agentic AI has the potential to disrupt almost every business process prevalent in an enterprise today.

In this article, we show how agentic AI can be applied to **industrial IoT** environments in manufacturing facilities, buildings, and factories. Industrial IoT focuses on the automation and monitoring of physical assets, e.g., compressors,

In this context, we introduce an agentic AI system designed to efficiently query historical / real-time sensor data from industrial IoT systems using natural language. For example, a query such as:

"How much power was B2 HVAC 2-1-1 using on 10th Aug 2025 at the Swiss site?"

illustrates the agent's interaction mode.

Even such short queries require reasoning over multiple domain concepts, e.g., assets, sensors, timestamps, and site locations — highlighting the need for domain-aware and context-sensitive agents.

Towards this end, we first outline the reference **architecture** of an agentic AI platform. We then outline the task-specific agents and tools relevant for a manufacturing / industrial IoT environment, and show how it can orchestrate those agents with introspection based reasoning models.

2. Agentic AI Reference Architecture

- **Reasoning** module: to decompose complex tasks and adapt their execution to achieve the given objective;
- **Agentic marketplace**: of existing and available agents;
- **Orchestration** module: to orchestrate and monitor (observe) the execution of multi-agent systems;
- **Integration** module: to integrate with enterprise systems, e.g., SCADA, KB repositories;
- Shared **memory** management for data and context sharing among agents;
- **Governance** layer, including explainability, privacy, security, safety guardrails, etc.

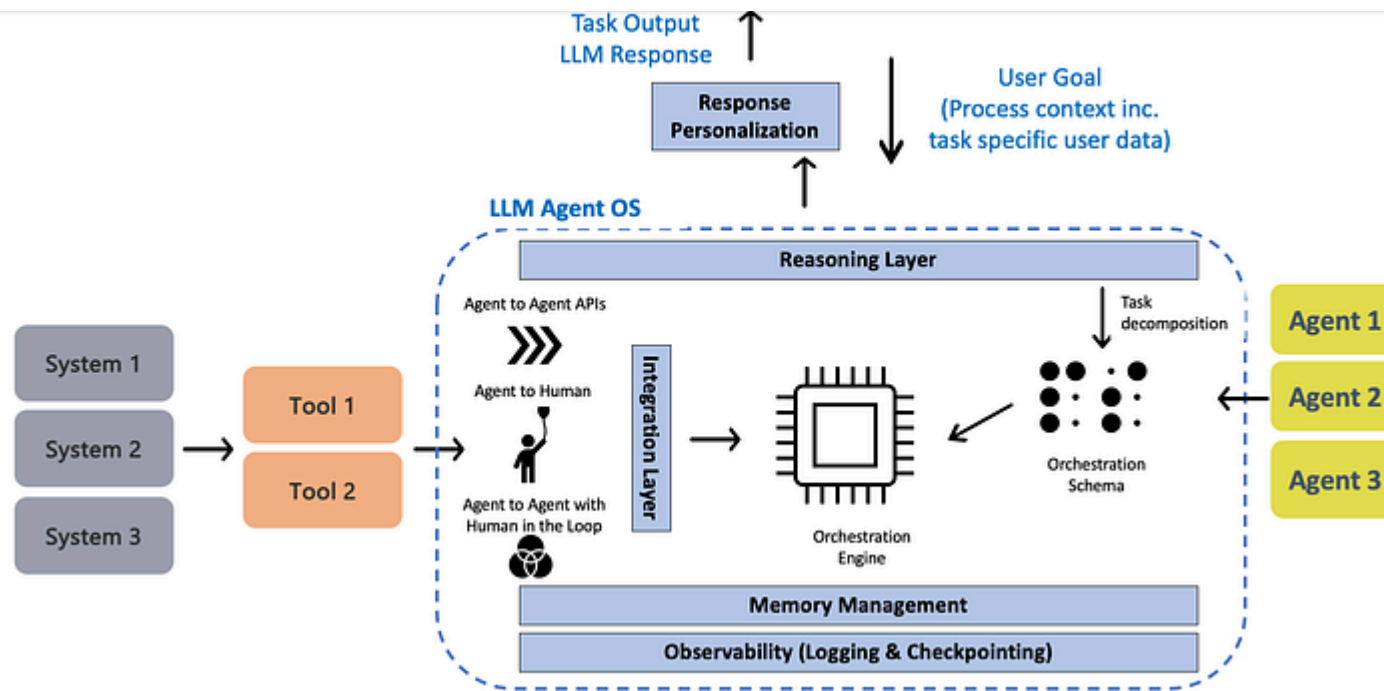


Fig. 2: Agentic AI Platform Reference Architecture (Image by Author)

Given a user task, the goal of the agentic AI platform is to identify (compose) an agent (group of agents) capable to executing the given task. So the first component we need is a **reasoning** module capable of decomposing a task into sub-tasks, with execution of the respective agents orchestrated by an orchestration engine.

A high-level approach to solving such complex tasks involves: (a) **decomposition** of the given complex task into (a hierarchy or workflow of) simple tasks, followed

complex user task, the system comes up with a plan to fulfill the request depending on the capabilities of available agents at run-time. In the static approach, given a set of agents, composite agents are defined manually at design-time combining their capabilities.

Chain of Thought (CoT) is the most widely used decomposition framework today to transform complex tasks into multiple manageable tasks and shed light into an interpretation of the model's thinking process.

Further, the **ReAct** (reasoning and acting) framework allows an agent to critically evaluate its own actions and outputs, learn from them, and subsequently refine its plan / reasoning process.

Agent composition implies the existence of an agent marketplace / registry of agents — with a well-defined description of the agent capabilities and constraints. For example, the Agent2Agent (A2A) protocol specifies the notion of an Agent Card (a JSON document) that serves as a digital "business card" for agents. It includes the following key information:

Copy

A2A Capabilities: Supported protocol features like streaming or pushNotifications
Authentication: Required authentication schemes (e.g., "Bearer", "OAuth2") to
Skills: A list of specific tasks or functions the agent can perform (AgentSkills)

Given the need to orchestrate multiple agents, there is a need for a **system integration module** supporting different agent interaction patterns, e.g., agent-to-agent API, agent API providing output for human consumption, human triggering an AI agent, AI agent-to-agent with human in the Loop. The integration patterns need to be supported by the underlying Agent OS platform.

Andrew Ng recently talked about this aspect from a performance perspective:

Today, a lot of LLM output is for human consumption. But in an agentic workflow, an LLM might be prompted repeatedly to reflect on and improve its output, use tools, plan and execute multiple steps, or implement multiple agents that collaborate. So, we might generate hundreds of thousands of tokens or more before showing any output to a user. This makes fast token generation very desirable and makes slower generation a bottleneck to taking better advantage of existing models.

Given the long-running nature of such complex tasks, memory management is key for agentic AI systems.

This entails both context sharing between tasks and maintaining execution context over long periods.

The standard approach here is to save the embedding representation of agent information into a vector store database that can support maximum inner product search (MIPS). For fast retrieval, the approximate nearest neighbors (ANN) algorithm is used that returns approximately top k-nearest neighbors with an accuracy trade-off versus a huge speed gain.

Finally, the **data governance module**. We need to ensure that data shared by the user specific to a task, or user profile data that cuts across tasks; is only shared with the relevant agents (table / report authentication and access control). Refer to my previous article on Responsible AI Agents for a discussion on the key dimensions needed to enable a well governed AI agent platform in terms of hallucination guardrails, data quality, privacy, reproducibility, explainability, human-in-the-loop (HITL), etc.

3. Agentic AI Architecture for Industrial IoT

In this section, we adapt the reference agentic AI platform (presented above) to one capable of serving industrial environments. APIs can be used to retrieve both structured and unstructured data, and a reasoning large language model (LLM) can then effectively utilize the data to make decisions / decide on the next step(s).

More specifically, the proposed **multi-agentic system** provides the below three core capabilities: contextual reasoning, specialized agents, tools integration—illustrated in Fig. 3.

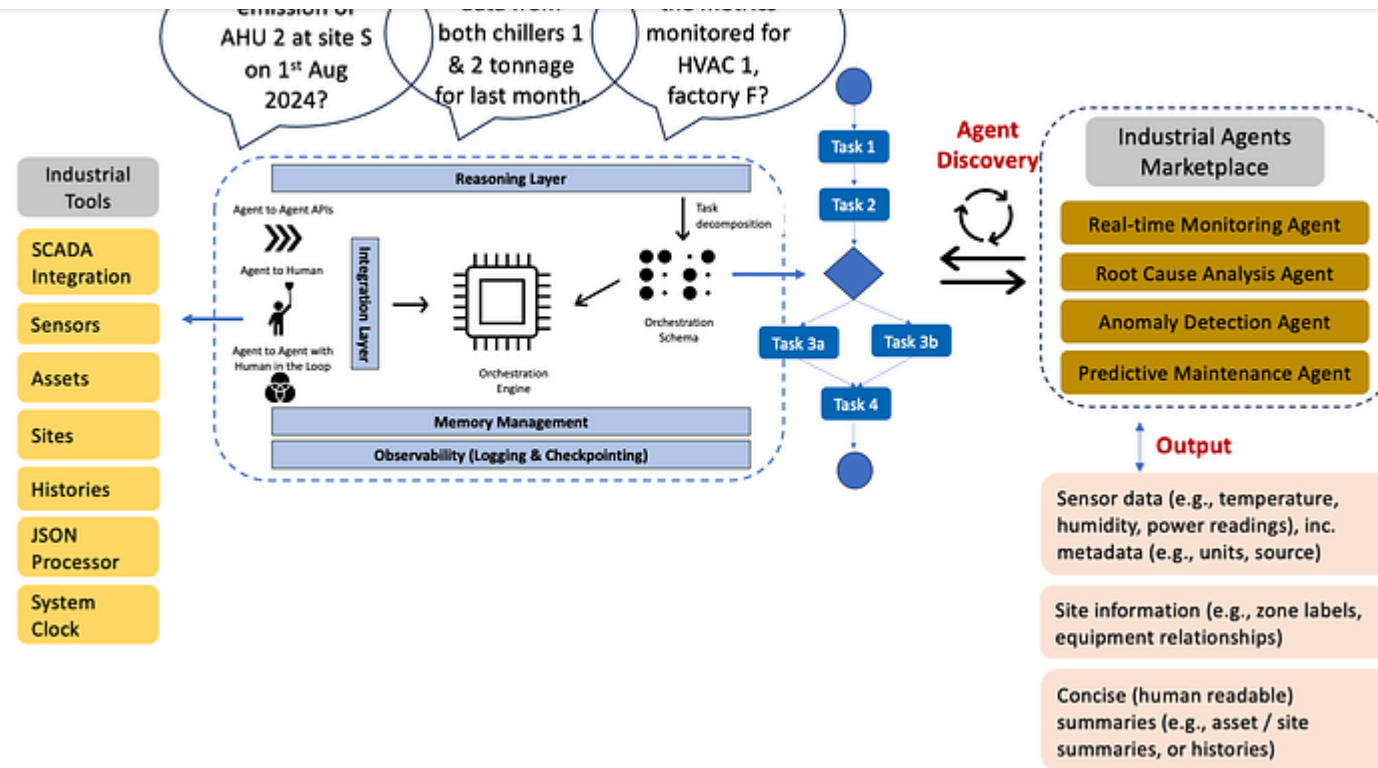


Fig. 3: Agentic AI Reference Architecture for Industrial IoT (Image by Author)

Contextual reasoning: Handling domain-specific reasoning tasks, e.g.

- sensor disambiguation,
- entity grounding, and
- statistical reasoning tasks, e.g., last week, max value.

language queries and generate actionable insights for industrial data retrieval and monitoring / predictive maintenance.

Specialized agents: catering to the industrial IoT domain, e.g.

- real-time monitoring agent,
- root-cause analysis agent,
- anomaly detection agent,
- predictive maintenance agent.

The agents use tools to query both historical and real-time data for operational metrics, e.g., energy usage, tonnage, temperature differentials, and sensor health.

Tools integration: Support for complex, multi-parameter industrial tools. These include four specialized tools for

- external data access (sensors, assets, sites, histories), and
- two internal utilities (JSON processor, clock — return the current time).

arguments robustly.

Tools primarily retrieve and output:

- sensor data (e.g., temperature, humidity, power readings), including metadata (e.g., units, source), and
- site information (e.g., zone labels, equipment relationships).

The agents then either return these (raw) data points directly or build upon them to produce concise (human readable) summaries, e.g., asset / site summaries, or histories.

3.1 ReAct Reasoning with Introspection

While standard ReAct based agents are effective for web retrieval type of tasks, it has been shown to be insufficient for industrial IoT environments, often exhibiting:

- gaps in domain-specific reasoning, e.g., linking chiller unit tonnage with energy efficiency — a vital linkage in industrial IoT environments.
- inconsistent reasoning, e.g., in date-offset reasoning ("last-day / week / month")

To overcome these challenges, we enhance the agent(s) with an iterative ReAct + **introspection** strategy — enabling the agentic system to handle complex, industrial domain-specific queries. The introspection strategy proceeds as follows — illustrated in Fig. 4.

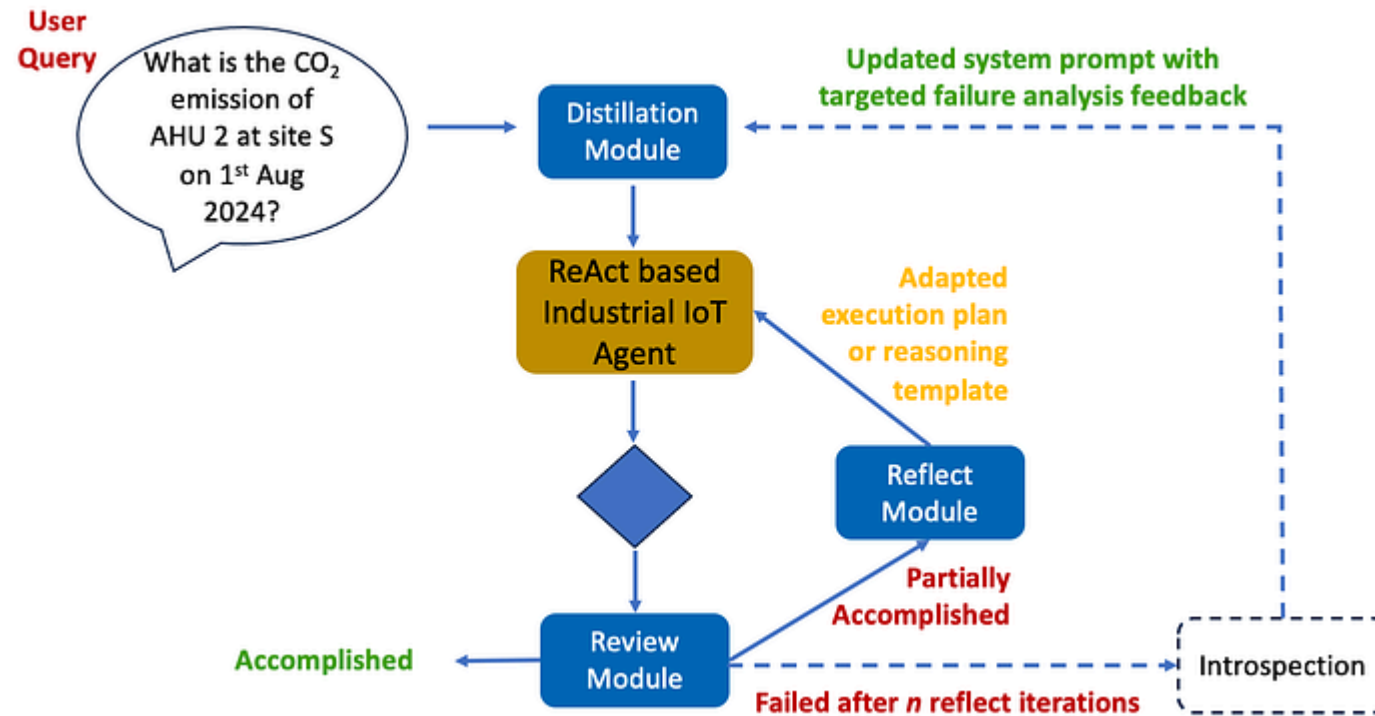


Fig. 4: ReAct based agentic reasoning with Introspection (Image by Author)

the underlying orchestration framework, generating an execution plan in response to the given user query.

To enhance reasoning fidelity, esp. for entity disambiguation, the orchestrator emits an internal **sub-query** (before initiating execution) to guide subsequent reasoning.

This anticipatory query is also answered by the LLM and improves plan coherence, task adherence, and tool invocation accuracy — sample system prompt below.

Copy

You are an advanced reasoning agent that can improve based on introspection. You will be given a previous reasoning trial in which you were given access to multiple agents and tools and a query to answer.

You were unsuccessful in resolving the query correctly either because you misunderstood the query, or you used up your set number of reasoning steps.

In a few sentences, diagnose a possible reason for failure and devise a new high-level execution plan that aims to mitigate the same failure. Use complete sentences.

```
Previous trial:  
Query: {query}  
{plan}
```

The **review** module acts as an LLM-as-a-Judge verifier classifying the output of the final reasoning step as

- accomplished,
- partially accomplished, or
- failed;

based on whether the generated output addressed the user query. This triggers the **reflect** module that performs introspection of the execution plan, assessing reasoning steps, agent / tool invocations, etc. It outputs

targeted feedback in the form of execution plan shifts or reasoning templates that are added to the system prompt to guide future executions.

4. Conclusion

pervasive industrial IoT environment in manufacturing sector.

The manufacturing industry is considered a highly conservative one with a wait-and-watch approach towards technological advancements, esp., AI. We showed that generative and agentic AI can add significant value to industrial IoT environments today, reimagining traditional manufacturing processes around real-time asset monitoring, root-cause analysis, and predictive maintenance.

Towards this end, we identified the industrial task-specific agents and tools, integrated them into a reference agentic platform to seamlessly answer user queries in natural language. We further showed how the (default) ReAct based agentic framework can be extended with 'introspection' to address domain related reasoning challenges with respect to, e.g., entity disambiguation.

We believe that the work will act as a significant enabler in driving business value by agentification (in the otherwise slow to change) manufacturing sector.

[#agentic-ai](#) [#agentic-ai-systems](#) [#industrial-iot](#) [#manufacturing](#) [#reasoning](#)